# Deeparture V2 - Phase 2B

**Admin Automation & Management Enhancements - Developer Specification Rev 1.3**

**Revision 1.3:** preserves Phase 2B scope and adds implementation guardrails for migrations, indexes, pagination, concurrency, API consistency, traceability, bulk safety, retention, performance and regression testing.

## Admin Automation & Management Enhancements

**Revision 1.3 - Architecture-aligned + implementation guardrails after completion of Phase 2A.**

This revision preserves the Phase 2B business scope, retains the architecture guardrails introduced in Rev 1.2, and adds implementation safeguards for migrations, indexing, pagination, concurrency, API consistency, traceability, bulk processing, data growth, performance and regression safety.

Phase 2B builds on the accepted Phase 2A baseline. Do not rebuild working Phase 2A services.

Add the advanced Admin capabilities that improve scale, traceability, automation readiness and day-to-day management while preserving the shared V2 architecture.

Phase 2B should improve operational control without changing the core business flows already proven in Phase 2A.

The main architectural rule is:

*Every new Phase 2B capability should extend the shared Deeparture services. Do not create parallel notification, search, source-mapping, availability, pricing, catalogue or audit systems that later phases would need to replace.*

Phase 2B assumes the following Phase 2A foundations are complete and remain authoritative:

- dedicated Admin V2 access and permission separation
- catalogue moderation and publication states
- normalized catalogue import pipeline with preview before commit
- canonical locations / destination / port references
- extensible typed attributes
- operator organisations and claim workflow
- Admin-first enquiry workflow
- normalized Availability Service as the single availability write boundary
- Phase 2A audit foundations
- responsive Admin V2 foundations
- Render staging deployment with a dedicated V2 Atlas staging database

If Phase 2B uncovers a defect in one of these foundations, fix the existing service rather than introducing a second implementation.

Main additions:

- richer import history
- advanced import error reporting
- advanced duplicate reconciliation
- reusable in-app notification service and Admin notification centre
- richer searchable audit log
- source/provenance management
- richer verification controls
- claim invitation history
- configurable listing completeness service
- advanced organisation activity
- website source queue
- advanced Admin dashboard metrics

- safe bulk actions through domain services
- advanced Admin filtering through the shared Search Service
- typed-attribute filtering validation
- availability source/freshness management
- stale-availability warnings
- availability update history
- manual availability overrides
- multi-source conflict/priority handling
- reusable external source / provider mapping foundation
- connector-ready source configuration
- lightweight event/background-job readiness
- operational reporting
- database migration and rollback discipline
- server-side pagination/sorting/limits for growing operational datasets
- query/index planning for Phase 2B access patterns
- concurrency/lost-update protection
- consistent API error contract
- operation/correlation IDs for imports, bulk jobs and sync/source operations
- bounded bulk operations with per-record outcomes
- data growth/retention readiness
- realistic-data performance verification
- full Phase 1 + Phase 2A regression gate

## Step 1 - Advanced Import History

Build:

```
/admin/v2/imports/history
```

Each batch should show:

- timestamp
- Admin user
- source / filename
- source type
- adapter type
- import schema/template version
- import type
- row total
- created
- updated
- skipped
- duplicates
- errors
- status
- affected records
- batch ID

Suggested statuses:

```
uploaded
validated
awaiting_confirmation
processing
completed
completed_with_errors
failed
cancelled
```

### Architecture requirement

Import history must reference the Phase 2A normalized import pipeline. Do not create a second import engine for history/retry tooling.

Recording `adapterType` and `schemaVersion` is required so future API/feed/spreadsheet versions can be traced to the contract that produced a batch.

## Step 2 - Import Error Reports

Allow Admin to:

- open row-level errors
- filter by error type
- download error report
- open affected source record
- retry corrected rows where practical

Retries must remain idempotent and must pass through the same normalized validation/matching/commit pipeline.

## Step 3 - Duplicate Reconciliation Workspace

Create a review UI for ambiguous matches.

Admin can:

- compare incoming organisation/listing vs existing record
- choose match existing
- choose create new
- merge selected fields where appropriate
- mark as not duplicate

Never auto-merge ambiguous data.

Where a reconciliation decision should influence future matching, store the decision/mapping in a reusable source/mapping structure rather than hidden UI-only state.

## Step 4 - Reusable Notification Service & Admin Notification Centre

Create/extend a Notification service and Admin UI.

Examples:

- new listing submitted
- claim completed
- import completed
- import failed
- enquiry received
- operator accepted enquiry
- listing needs review
- stale availability warning
- source/integration failure

Minimum fields should support future recipients, not only a global Admin inbox:

```
type
title
message
recipientUserId optional
recipientOrganisationId optional
audience admin | operator | system
relatedRecordType
relatedRecordId
actionUrl / actionTarget optional
readAt
createdAt
```

Admin header should show unread count.

### Architecture requirement

Phase 2B may expose only the Admin notification centre, but the underlying service must be recipient-aware so Phase 3 can reuse it for Operator notifications.

Do not build a second Operator notification subsystem in Phase 3.

Email remains a notification channel/service integration; notification business events should not be tightly coupled to email-specific controller logic.

## Step 5 - Advanced Audit Log

Build searchable audit UI:

```
/admin/v2/audit
```

Track at minimum:

- publish/unpublish
- restrict/reactivate/archive
- claim invitation
- claim completion
- import commit
- duplicate reconciliation decision
- attribute changes
- destination/port changes
- enquiry status changes
- organisation changes
- availability override
- availability resolution changes where material
- verification changes
- bulk actions

Audit fields:

```
actorUserId
actorOrganisationId
action
entityType
entityId
before
after
metadata
createdAt
```

Never store passwords, raw claim tokens, API credentials or secrets in audit payloads.

## Step 6 - Provenance / Source Management

Add richer source tracking that is reusable across catalogue and availability workflows.

At minimum, support:

```
sourceType
sourceRef / providerRef
sourceUrl
sourceName
lastCheckedAt
lastUpdatedAt
```

Suggested source types:

```
deeparture_import
admin_manual
operator
operator_website
csv
xlsx
api
webhook
feed
legacy_migration
```

Field-level provenance can remain a later enhancement if it adds excessive complexity, but Phase 2B should not prevent it.

### Reusable external source mapping

The mapping concept must be platform-wide rather than availability-only.

A mapping should be able to express:

```
source/provider
internalEntityType
internalEntityId
externalEntityType
externalId
organisationId optional
metadata optional
createdAt
updatedAt
```

This allows one Deeparture entity to have multiple external provider IDs without changing its permanent Deeparture identity.

The same mapping foundation should later support catalogue imports, supplier APIs, availability feeds and booking-system connectors.

## Step 7 - Verification Controls

Verification remains separate from publication state.

Admin can:

- mark organisation/listing verified
- see operator verification state
- remove/revoke verification when information becomes stale
- see who performed the verification
- see when verification occurred
- see the verification source/scope where relevant

Recommended metadata:

```
verificationStatus
verifiedByUserId
verifiedAt
verificationSourceType
verificationSourceRef
verificationScope
verificationNotes optional
```

Avoid treating verification as a permanent yes/no flag with no provenance.

## Step 8 - Claim Invitation Management

Add:

- invitation history
- sent date
- expiry
- resend
- revoke
- claimed date
- invited email
- status

Admin should see all prior claim attempts.

Reuse the secure Phase 2A claim service and token model. Do not create a second claim mechanism.

## Step 9 - Configurable Listing Completeness Service

Add listing completeness indicators.

Example categories:

- core identity
- images
- facilities
- dietary
- accessibility
- destinations
- pricing
- availability
- contact/ownership

Example result:

```
Damai I - 82% complete
Missing:
- hero image
- accessibility details
- current departures
```

Completeness must be informational, not a requirement for basic publication.

### Architecture requirement

Completeness rules must not live only as hard-coded frontend checks.

Use a shared/configurable completeness service or rule definition by product type so the same result can later be shown in:

- Admin V2
- Operator Portal
- reporting
- API responses where appropriate

## Step 10 - Advanced Organisation Activity

Organisation page can show:

- users
- vessels
- resorts
- enquiries
- claim history

- imports affecting organisation
- verification
- recent changes
- last login/activity
- notifications/issues requiring attention
- source/provider mappings where relevant

## Step 11 - Website Source Queue

Build operational queue:

```
/admin/v2/imports/website
```

Admin can enter:

- organisation/operator
- product type
- source URL
- product name
- notes

Statuses:

```
queued
draft_created
needs_review
approved
rejected
```

Phase 2B should make this ready for Phase 5 automated extraction workers.

A sophisticated scraper/AI extractor is not required in Phase 2B.

Any later extractor must feed the normalized import service rather than write directly into catalogue models.

## Step 12 - Availability Source Management & Freshness

Build the management layer on top of the normalized Availability Service created in Phase 2A.

Phase 2B still does **not** need to connect to external providers. The goal is to make the availability system ready for those connectors.

### Admin visibility

For departure/cabin or resort availability, Admin should be able to see:

```
effective availability status / quantity
source type
source name/reference
sourceUpdatedAt
receivedAt
freshness state
manual override state
conflict state where applicable
```

### Freshness / stale rules

The system must distinguish between:

```
fresh
aging
stale
unknown
```

The threshold should be configurable by source/source type rather than hard-coded into public-page components.

A stale value must **not automatically become sold out**.

Public/search APIs should be able to return freshness metadata so later customer UI can display appropriate language.

## Source configuration foundation

Create a connector/source configuration suitable for later Phase 5 integrations.

Suggested non-secret metadata:

```
providerName
sourceType
organisationId
external mapping references
enabled
priority
freshnessThresholdMinutes
lastSuccessfulSyncAt
lastAttemptAt
status
```

Where possible, provider/entity IDs should use the shared external source mapping foundation from Step 6 rather than creating availability-only mapping tables.

Do not store provider API credentials in catalogue records, source records or audit logs.

## Multiple sources / conflicts

The architecture must allow more than one source to report availability for the same inventory.

Phase 2B should provide a deterministic and reviewable resolution strategy:

- configurable source priority
- flag conflicting values
- allow Admin to inspect competing source updates
- do not silently replace a higher-priority/current accepted source with older/lower-priority data

Do not hard-code a universal provider ranking.

## Manual override

Admin should be able to apply an explicit availability override with:

```
value/status
reason
createdBy
createdAt
optional expiresAt
```

The UI must make clear when effective availability is being driven by a manual override.

An expired override must stop controlling the effective value.

## Availability history

Retain enough history to answer:

```
What did the source report?
When did it report it?
What became the effective availability?
Was there a manual override?
Why did the effective value change?
```

History does not need to become an analytics warehouse, but important inventory changes must be traceable.

## Phase 5 connector readiness

Future connectors must write through the existing Phase 2A Availability Service.

Examples:

```
specialist liveaboard availability API
booking platform API
operator proprietary API
JSON/XML feed
webhook
Google Sheet
SFTP/file feed
```

## Step 13 - Advanced Dashboard

Add metrics such as:

- listings needing review
- stale listings
- imports with errors
- unclaimed operators
- expired invitations
- incomplete listings
- enquiries awaiting Admin
- recently claimed organisations
- stale availability
- availability conflicts
- manual overrides
- source/integration errors

## Step 14 - Safe Bulk Admin Actions

Where safe, support:

- bulk publish
- bulk restrict
- bulk archive
- bulk assign destination
- bulk send invitations
- bulk mark for review

Bulk actions must show confirmation and results.

### Architecture requirement

Bulk actions must call the same domain/service methods used by individual actions so that:

- permissions are enforced
- state transitions remain valid
- audit records are created
- notifications/events are triggered consistently

Do not implement bulk actions as direct MongoDB mass updates that bypass business rules.

### Bulk safety requirement

Bulk actions must be bounded and observable:

- apply a sensible maximum batch size or process large jobs in bounded chunks
- return per-record success/failure/skipped results where practical
- assign an operation/correlation ID to the bulk action
- make partial failure visible rather than reporting one opaque success/failure
- keep retries idempotent where an operation may be repeated

## Step 15 - Advanced Filters Through the Shared Search Service

Admin catalogue filters can include:

- operator

- publication status
- claim status
- verification
- destination
- country
- departure/arrival port where relevant
- source
- completeness
- last updated
- has availability
- availability freshness
- availability source
- availability conflict/override state
- has pricing
- effective price range
- has images

## Typed attribute filtering requirement

Phase 2B must prove that the shared Search Service can combine first-class structured fields with filterable typed attributes.

Support/search semantics should be demonstrated for the Phase 2A attribute types in use, such as:

- boolean
- number
- controlled single-select / enum
- controlled multi-select

Examples of future attributes include:

- vegan grade
- accessibility support/level
- technical diving support
- rebreather support
- nitrox
- photography facilities
- minimum logged dives

Phase 2B does **not** need to build the final customer filter UI. That remains Phase 4.

However, the backend/search layer must prove that combined queries work without adding product-specific hard-coded query logic for every new attribute.

Example acceptance query:

```
country = Indonesia
AND departure port = Sorong
AND effective price <= 4000 USD-normalized
AND availability = available
AND vegan_grade >= 4
AND nitrox = true
AND technical_diving_support = true
```

The exact test data may differ, but the combined structured + typed-attribute behavior must be demonstrated.

## Query execution requirement

Search/filtering must remain server-side/database-driven for operationally significant result sets.

Phase 2B list/search APIs should support:

- pagination
- sorting
- explicit page/limit or cursor semantics
- bounded maximum page size
- total/count metadata where practical
- stable ordering for pagination

Do not fetch the entire catalogue, audit history, notification history or import history into the browser simply to filter/sort it there.

## Step 16 - Operational Reporting

Provide operational summaries such as:

- catalogue growth
- claimed vs unclaimed
- published vs draft
- import success/error rate
- listings not checked recently
- enquiries by status
- stale availability count
- availability sources requiring attention
- recent sync/update failures
- incomplete listings by category
- verification age/staleness

Keep this operational, not a full BI platform.

## Step 17 - Lightweight Event / Background-Job Readiness

Phase 2B should keep side effects behind service boundaries and prepare for asynchronous work without requiring a full message broker/event bus.

Useful internal event concepts include:

```
listing.published
listing.updated
import.completed
import.failed
claim.completed
enquiry.created
enquiry.approved
availability.updated
availability.conflict_detected
verification.changed
```

These events may initially be implemented with a lightweight internal dispatcher/service pattern.

The goal is to prevent future background tasks and notification channels from being tightly embedded inside controllers.

Future Phase 5/background workers may use this boundary for:

- connector sync
- retry handling
- stale-source checks
- webhook processing
- email delivery
- search indexing

A distributed event bus is a non-goal for Phase 2B unless genuinely required by the deployed architecture.

# Step 18 - Engineering Guardrails for Scale, Safety & Maintainability

Phase 2B introduces operational datasets and write paths that will grow over time. The following safeguards are required now so later phases can extend the platform without reworking these foundations.

## 18.1 Database migrations and rollback

Any schema/index/data migration introduced by Phase 2B must be documented.

For each migration, record:

```
purpose
affected collections/models
forward migration steps
whether existing records need backfill
compatibility considerations
rollback / recovery approach
```

Prefer additive/backward-compatible changes where practical.

Do not perform destructive staging/production data changes manually without a documented migration/recovery path.

## 18.2 Indexing plan

Indexes should reflect actual Phase 2B query patterns rather than being added indiscriminately.

Likely candidates include combinations around:

- notifications: recipient/audience + read state + createdAt
- audit: entityType/entityId + createdAt; actor + createdAt
- import history: batch ID, status, source/adapter + createdAt
- source/provider mappings: provider/source + external ID; internal entity
- availability source/history: inventory identity + source + timestamps
- freshness/status/override fields used by Admin filters
- canonical location/search fields used frequently
- publication/claim/verification states used in Admin filtering

Document important new indexes and the query/use-case each supports.

## 18.3 Pagination, sorting and bounded reads

The following growing datasets must be server-paginated or cursor-paginated:

- import history/errors
- notifications
- audit log
- claim history
- availability history
- source/integration history
- large catalogue/organisation lists

APIs must enforce sensible maximum page sizes.

## 18.4 Concurrency / lost-update protection

Phase 2B must not silently allow older writes to overwrite newer accepted data where concurrency is material.

Examples include:

- Admin edits vs import commit
- two Admin edits
- import vs manual override

- future source update vs manual/accepted value

Use an appropriate strategy such as:

- document version / revision
- updatedAt precondition
- optimistic concurrency control
- explicit conflict response
- deterministic source timestamp/priority rules for availability

A conflict should be visible and reviewable rather than silently resolving by "last request wins" when that could lose newer data.

## 18.5 Consistent API error contract

New Phase 2B APIs should use a predictable error shape.

At minimum distinguish:

```
validation_error
authentication_error
permission_error
not_found
conflict
rate_or_limit_error
internal_error
```

Include:

- stable error code
- human-readable message
- field/row details where useful
- operation/correlation ID where available

Do not expose stack traces or secrets to clients.

## 18.6 Operation / correlation IDs

Assign a stable operation ID to multi-step operational work such as:

- import batches
- bulk actions
- duplicate reconciliation operations where multi-step
- future sync/source attempts

The same ID should be usable to correlate:

- API response
- Admin UI/history
- audit entries
- logs
- error reports

This is an observability aid and must not contain secrets.

## 18.7 Data growth and retention readiness

Do not hard-code the assumption that every operational record must remain in the hot working set forever.

Phase 2B should distinguish conceptually between:

- business/audit records that may require long-term retention
- operational notifications
- technical sync/source attempt logs

- generated error/export artifacts

Exact production retention periods can be decided later, but records should be structured so archival/purge policies can be introduced safely without damaging catalogue/inventory state or required audit history.

## 18.8 Performance acceptance

Before Phase 2B sign-off, run the main Admin/search paths against a realistic seeded dataset rather than only a handful of demo records.

The objective is not a formal enterprise load test.

Verify that:

- combined search/filter queries remain database/service driven
- pagination does not degrade into full collection/browser loads
- common Admin list views remain responsive
- no obvious N+1 query pattern is introduced
- availability/history/audit views use bounded queries
- relevant indexes are used/available for the intended query patterns

Record the approximate seeded dataset size used for this check.

## 18.9 Regression gate

Phase 2B acceptance requires both:

1. Phase 2B acceptance tests pass, and
2. the existing Phase 1 + Phase 2A regression/verification suite remains green.

At minimum re-check:

- authentication/permissions
- organisations/claiming
- normalized catalogue import preview/confirm/commit
- canonical locations
- typed attributes
- pricing/effective price
- normalized Availability Service
- Admin-first enquiry workflow
- public/search APIs
- automatic liveaboard/resort pages

Do not sign off Phase 2B if a new operational feature breaks an accepted earlier-phase business flow.

## 18.10 Heavy infrastructure remains a non-goal

Do not introduce infrastructure solely for architectural fashion.

Phase 2B does not require:

- Redis
- Elasticsearch/OpenSearch
- Kafka or another distributed event broker
- Kubernetes
- microservice decomposition
- a dedicated enterprise job platform

The current Node/Express + MongoDB architecture may continue to be used.

Keep service boundaries clean so specialized infrastructure can be added later if actual scale or operational evidence justifies it.

Phase 2B must retain the responsive rules from Phase 2A.

Notification centre, duplicate review, audit search, verification, operator management and routine availability review should work on mobile.

Complex reconciliation/import analysis may remain easiest on desktop.

No critical Admin workflow should become unusable on phone.

Phase 2B is complete when:

- Import history is searchable and records source/adapter/schema-version context.
- Row-level import errors are visible.
- Ambiguous duplicates can be manually reconciled.
- Notification centre works through a recipient-aware reusable Notification service.
- Unread count works.
- Audit log records important actions and excludes secrets.
- Provenance/source information is visible.
- Reusable external source/provider mappings can be stored without replacing Deeparture internal IDs.
- Verification includes actor/time/source or scope metadata and remains separate from publication.
- Claim invitation history works using the existing claim service.
- Completeness indicators are generated by a shared/configurable service rather than frontend-only rules.
- Website source queue works and is ready to feed the normalized import service later.
- Admin can see availability source and last-update/freshness information.
- Stale availability is flagged and is not silently converted to sold out.
- Source priority can be configured for availability.
- Conflicting source values can be identified for review.
- Manual availability override works and is visibly distinguishable.
- Availability update/effective-value history is traceable.
- Connector/source configuration exists without requiring an external provider integration.
- Phase 2A normalized Availability Service remains the single write path for future adapters.
- Advanced dashboard highlights operational issues.
- Safe bulk actions use domain services and preserve permissions/state/audit behavior.
- Shared Search Service supports advanced structured filters.
- Shared Search Service can combine structured filters with typed filterable attributes.
- Operational reporting is available.
- Growing Admin/history datasets use server-side pagination/sorting with bounded page sizes.
- Important Phase 2B query patterns have documented database indexes.
- Phase 2B migrations/backfills have documented forward and rollback/recovery steps.
- Material concurrent updates cannot silently overwrite newer accepted data.
- New Phase 2B APIs follow a consistent error contract including conflict handling.
- Imports/bulk/multi-step operations expose operation/correlation IDs for traceability.
- Bulk actions are bounded and report per-record/partial outcomes.
- Data structures remain compatible with future retention/archival policies.
- A realistic seeded-data performance check has been completed.
- Phase 1 + Phase 2A regression/verification checks remain green.
- Notification/audit/background side effects are service/event-ready rather than tightly coupled to controllers.
- Core Phase 2A workflows remain unchanged and stable.

- Mobile/tablet/desktop remain usable.
- No V1 production data is modified.

Demonstrate:

```
1. Run import with mixed valid/error/duplicate rows.
2. Review import history including adapter/source/schema version.
3. Open a row-level error.
4. Resolve an ambiguous duplicate.
5. Publish corrected record.
6. Confirm audit log event.
7. Confirm recipient-aware Admin notification created.
8. Mark listing verified and show verified-by/time/source/scope metadata.
9. View provenance and source/provider mapping data.
10. Review listing completeness from the shared completeness service.
11. Add source URL to website queue.
12. Create/configure a test availability source with a freshness threshold.
13. Submit two source updates for the same inventory and demonstrate priority/conflict handling.
14. Mark an old availability value stale without converting it to sold out.
15. Apply a temporary manual availability override and confirm it is clearly identified.
16. Review availability source/effective-value history.
17. Run a safe bulk action and confirm state rules + audit remain intact.
18. Run a combined search/filter using canonical geography/price/availability plus at least two typed attributes.
19. Confirm the combined search uses the shared Search Service rather than page-only filtering.
20. Confirm Phase 2A import, claim, enquiry, pricing and availability regressions remain green.
21. Confirm Render staging URLs, Atlas staging database, Git branch and exact deployed commit.
22. Demonstrate server-side pagination/sorting on at least one growing operational dataset.
23. Show/document Phase 2B indexes and the query patterns they support.
24. Demonstrate one conflict/lost-update protection case.
25. Show a consistent API conflict/validation error response including an operation/correlation ID where
applicable.
26. Run a bounded bulk action with at least one mixed outcome and show per-record results.
27. Run a realistic seeded-data combined search and confirm it does not load the full catalogue into browser
memory.
28. Confirm Phase 1 + Phase 2A regression/verification suite remains green.
```

Phase 2B is not fully signed off until the staging baseline is recorded.

Record:

```
Admin V2 URL
Frontend Render staging URL
Backend/API Render staging URL
MongoDB Atlas staging database name/environment
Git branch
Exact deployed commit hash
Deployment date
Acceptance-test result
Known issues/exceptions
Small deployment-specific changes
```

The developer should explicitly confirm:

> *This commit is the version currently deployed to the Deeparture V2 Render staging environment.*

Acceptance tests should be run against the deployed staging version, not only locally.

Not required:

- full automatic website scraping
- AI extraction pipeline
- actual external availability API/feed integrations
- sharc or other booking-system provider integration
- full customer-facing V2 redesign
- final customer filter UI
- full Operator Portal V2
- PWA
- Agent Platform
- native app
- distributed event bus / enterprise message broker

- full BI/data warehouse
- Redis solely for Phase 2B
- Elasticsearch/OpenSearch solely for Phase 2B
- Kafka/distributed event broker
- Kubernetes
- microservice decomposition

Those belong to later phases.

After Phase 2B passes acceptance testing, proceed to **Phase 3 - Operator Portal V2**.

Phase 3 should reuse:

- the recipient-aware Notification service
- the shared Search Service
- the typed attribute/filter definitions
- the completeness service
- the normalized catalogue import service
- the normalized Availability Service
- the shared source/provider mapping foundation
- audit/provenance services
- organisation/membership/permission services

Do not recreate these specifically for the Operator Portal.